

ICT703 – Module 7 Workshop

Learning Objectives

In this Workshop you will:

1. *Practice Functions*
2. *Focus on Testing*

Designing Tests for Dates

Given a Date module with the following methods:

```
def setDate(year, month, day)          # creates supplied date

def getDay(), getMonth(), getYear()    # returns an int

def addDays(days)                      # advances a date by days

def daysInMonth(month, year)          # returns an int

def isLeapYear(year)                  # returns true or false

def nextDay()                          # advances by 1 day
```

You may assume that `isLeapYear(year)` and `daysInMonth(month, year)` are independently tested and thus you may use them.

Your test plan states also that `addDays` should be efficient enough to add 1,000,000 days in a call.

Activity 1: Write the `addDays` function

You have been supplied a partial file: `MyDate.py` in Canvas Week 10 materials. For this week you must not use existing Python `DateTime` libraries and instead should use only methods in the supplied file, or those you write yourself.

1. What is the pseudo-code for the algorithm for an `addDays(days)` method that moves the current `Date` object forward in time by the given number of days. A negative value moves the `Date` backward in time.
2. Come up with a set of test values (inputs and expected outputs) for your `addDays` method to test its correctness, for each case identify why you needed this test (i.e. testing a leap year)
3. Code the `addDays(days)` function and implement your test cases in python. (Note you need to make your function efficient, using the minimum number of loop iterations etc.).